K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering & Technology
Lab Assignment

Course Name: Analysis and Design of Algorithms Lab
Course Code: ENCS256
Name: YUVRAJ SAINI
Semester: IV
Roll No. :- 2401350008
Github : [Yuvraj22-game/ADA_SEMESTER_4: semester 4 ada assignments]

Bubble Sort:

Code:

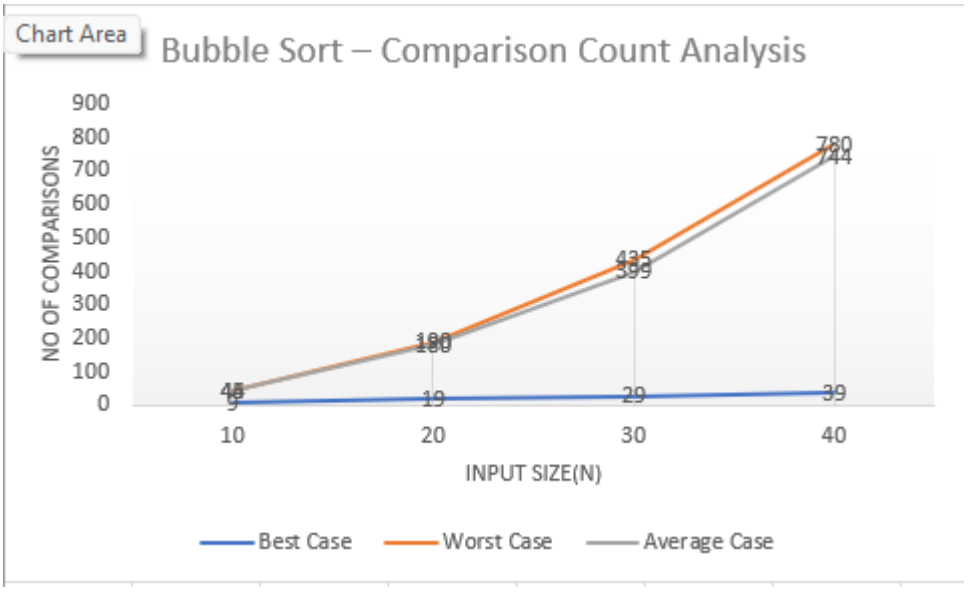
```
// This is a Java program for Bubble Sort algorithm with step counter
with arrays having 10 , 20, 30, 40 elements
import java.util.Random;

public class bubblesort {

    public static long bubbleSort(int[] arr) {
        long comparisons = 0;
        int n = arr.length;
        boolean swapped;

        for (int i = 0; i < n - 1; i++) {
            swapped = false;
            for (int j = 0; j < n - i - 1; j++) {
                comparisons++;
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = true;
                }
            }
            if (!swapped) break;
        }
        return comparisons;
    }

    public static int[] generateRandomArray(int n) {
        Random rand = new Random();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = rand.nextInt(1000);
        }
        return arr;
    }
```

```java
    public static int[] generateSortedArray(int n) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }
        return arr;
    }

    public static int[] generateReverseArray(int n) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = n - i;
        }
        return arr;
    }

    public static void main(String[] args) {

        int[] sizes = {10, 20, 30, 40};

        for (int size : sizes) {
            System.out.println("Input Size: " + size);

            long best = bubbleSort(generateSortedArray(size));
            long worst = bubbleSort(generateReverseArray(size));
            long avg = bubbleSort(generateRandomArray(size));

            System.out.println("Best Case Comparisons: " + best);
            System.out.println("Worst Case Comparisons: " + worst);
            System.out.println("Average Case Comparisons: " + avg);
            System.out.println("----------------------------");
        }
    }
}
```

Output :

```
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES
ssages' '-cp' 'C:\Users\yuvraj\AppData\Roaming\Code
b6392\bin' 'bubblesort'
-----------------------------
Input Size: 20
Best Case Comparisons: 19
Worst Case Comparisons: 190
Average Case Comparisons: 190
-----------------------------
Input Size: 30
Best Case Comparisons: 29
Worst Case Comparisons: 435
Average Case Comparisons: 425
-----------------------------
Input Size: 40
Best Case Comparisons: 39
Worst Case Comparisons: 780
Average Case Comparisons: 759
```

GRAPH:



Selection Sort :

CODES:

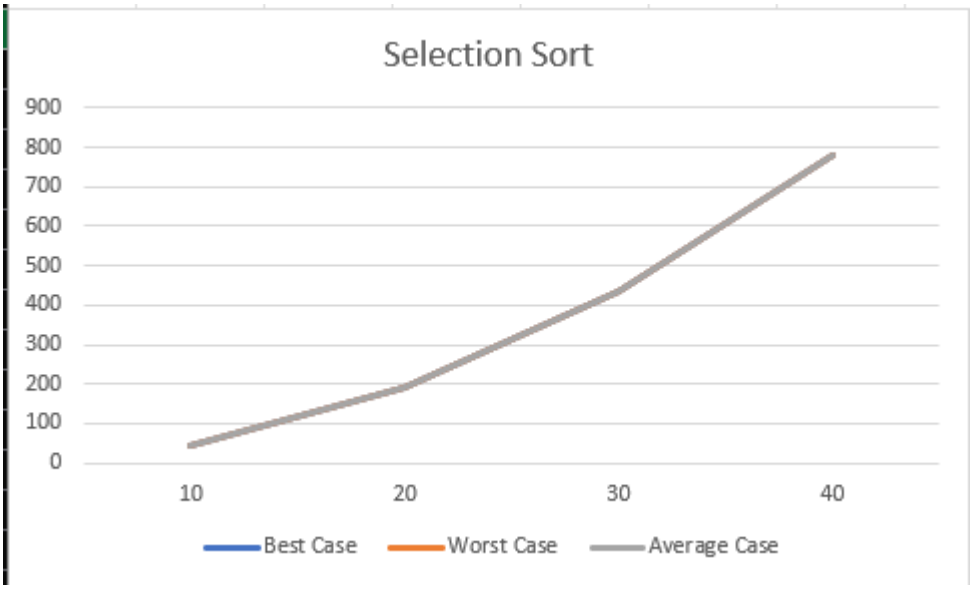
```java
import java.util.Random;

public class selectionsort {

    public static long selectionSort(int[] arr) {
        long comparisons = 0;
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            int minIndex = i;
            for (int j = i + 1; j < n; j++) {
                comparisons++;   // counting comparison
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }
            int temp = arr[minIndex];
            arr[minIndex] = arr[i];
            arr[i] = temp;
        }
        return comparisons;
    }

    public static int[] generateRandomArray(int n) {
        Random rand = new Random();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = rand.nextInt(1000);
        }
        return arr;
    }
```

```java
    public static int[] generateSortedArray(int n) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }
        return arr;
    }

    public static int[] generateReverseArray(int n) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = n - i;
        }
        return arr;
    }

    public static void main(String[] args) {

        int[] sizes = {10, 20, 30, 40};

        for (int size : sizes) {
            System.out.println("Input Size: " + size);

            long best = selectionSort(generateSortedArray(size));
            long worst = selectionSort(generateReverseArray(size));
            long avg = selectionSort(generateRandomArray(size));

            System.out.println("Best Case Comparisons: " + best);
            System.out.println("Worst Case Comparisons: " + worst);
            System.out.println("Average Case Comparisons: " + avg);
            System.out.println("----------------------------");
        }
    }
}
```

OUTPUT:

```
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>  & 'C:\P
ssages' '-cp' 'C:\Users\yuvraj\AppData\Roaming\Code\User\workspac
b6392\bin' 'selectionsort'
-----------------------------
Input Size: 20
Best Case Comparisons: 190
Worst Case Comparisons: 190
Average Case Comparisons: 190
-----------------------------
Input Size: 30
Best Case Comparisons: 435
Worst Case Comparisons: 435
Average Case Comparisons: 435
-----------------------------
Input Size: 40
Best Case Comparisons: 780
Worst Case Comparisons: 780
Average Case Comparisons: 780
-----------------------------
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>
```

GRAPH:



Insertion Sort

CODE:

```
// This is a Java program for insertion Sort algorithm with step
counter with arrays having 10 , 20, 30, 40 elements
```

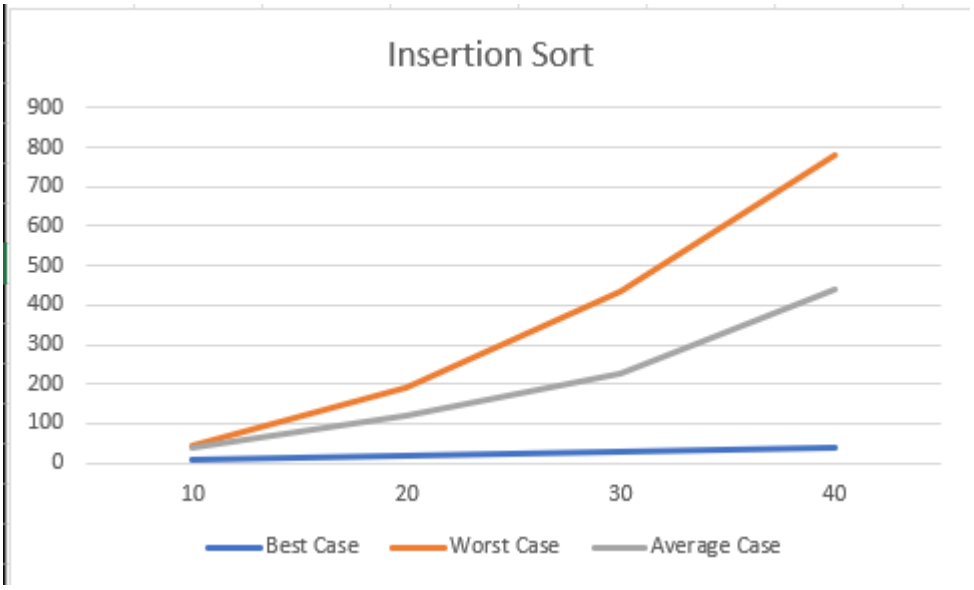
```
import java.util.Random;

public class insertionsort {

    public static long insertionSort(int[] arr) {
        long comparisons = 0;
        int n = arr.length;

        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;

            while (j >= 0) {
                comparisons++;   // counting comparison
                if (arr[j] > key) {
                    arr[j + 1] = arr[j];
                    j--;
                } else {
                    break;
                }
            }
            arr[j + 1] = key;
        }
        return comparisons;
    }

    public static int[] generateRandomArray(int n) {
        Random rand = new Random();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = rand.nextInt(1000);
        }
        return arr;
```

```java
    }

    public static int[] generateSortedArray(int n) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }
        return arr;
    }

    public static int[] generateReverseArray(int n) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = n - i;
        }
        return arr;
    }

    public static void main(String[] args) {

        int[] sizes = {10, 20, 30, 40};

        for (int size : sizes) {
            System.out.println("Input Size: " + size);

            long best = insertionSort(generateSortedArray(size));
            long worst = insertionSort(generateReverseArray(size));
            long avg = insertionSort(generateRandomArray(size));

            System.out.println("Best Case Comparisons: " + best);
            System.out.println("Worst Case Comparisons: " + worst);
            System.out.println("Average Case Comparisons: " + avg);
            System.out.println("----------------------------");
        }
    }
}
```

OUTPUT:

```
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>  & 'C:
ssages' '-cp' 'C:\Users\yuvraj\AppData\Roaming\Code\User\worksp
b6392\bin' 'insertionsort'
Input Size: 10
Best Case Comparisons: 9
Worst Case Comparisons: 45
Average Case Comparisons: 34
----------------------------
Input Size: 20
Best Case Comparisons: 19
Worst Case Comparisons: 190
Average Case Comparisons: 118
----------------------------
Input Size: 30
Best Case Comparisons: 29
Worst Case Comparisons: 435
Average Case Comparisons: 281
----------------------------
Input Size: 40
Best Case Comparisons: 39
Worst Case Comparisons: 780
Average Case Comparisons: 361
----------------------------
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>
```

GRAPH:



Linear Search

CODE :

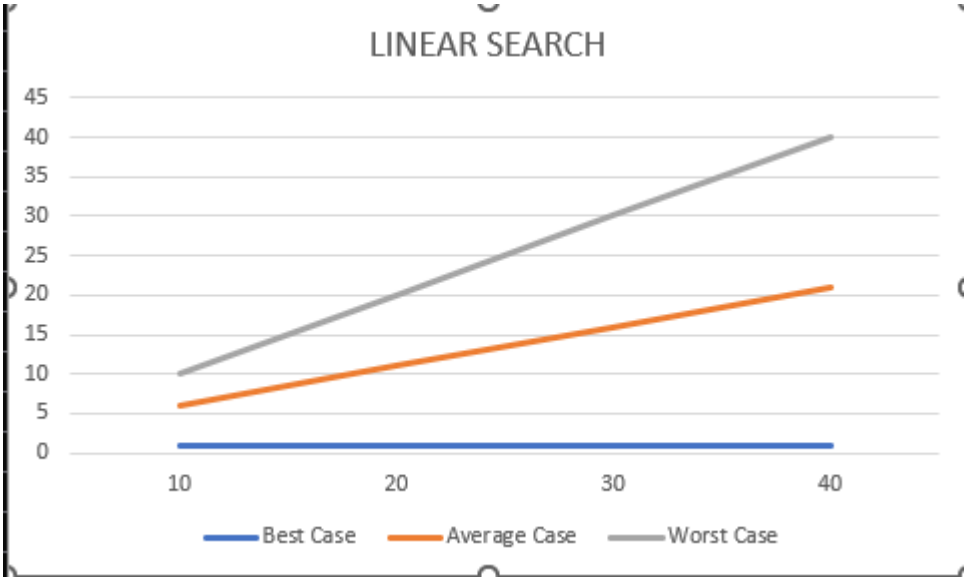
```
import java.util.Random;

public class linearsearch {

    public static long linearSearch(int[] arr, int key) {
        long comparisons = 0;

        for (int i = 0; i < arr.length; i++) {
            comparisons++;
            if (arr[i] == key) {
                return comparisons;
            }
        }
        return comparisons;
    }

    public static int[] generateRandomArray(int n) {
        Random rand = new Random();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = rand.nextInt(1000);
        }
        return arr;
    }

    public static int[] generateSortedArray(int n) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }
        return arr;
    }

    public static void main(String[] args) {
```

```java
        int[] sizes = {10, 20, 30, 40};

        for (int size : sizes) {
            int[] arr = generateSortedArray(size);

            System.out.println("Input Size: " + size);

            long best = linearSearch(arr, arr[0]);          // first
element
            long avg = linearSearch(arr, arr[size/2]);      // middle
element
            long worst = linearSearch(arr, -1);             // not
present

            System.out.println("Best Case Comparisons: " + best);
            System.out.println("Average Case Comparisons: " + avg);
            System.out.println("Worst Case Comparisons: " + worst);
            System.out.println("----------------------------");
        }
    }
}
```

OUTPUT:

```
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>  & 'C
ssages' '-cp' 'C:\Users\yuvraj\AppData\Roaming\Code\User\works
b6392\bin' 'linearsearch'
Input Size: 10
Best Case Comparisons: 1
Average Case Comparisons: 6
Worst Case Comparisons: 10
----------------------------
Input Size: 20
Best Case Comparisons: 1
Average Case Comparisons: 11
Worst Case Comparisons: 20
----------------------------
Input Size: 30
Best Case Comparisons: 1
Average Case Comparisons: 16
Worst Case Comparisons: 30
----------------------------
Input Size: 40
Best Case Comparisons: 1
Average Case Comparisons: 21
Worst Case Comparisons: 40
----------------------------
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>
```

GRAPH:


LINEAR SEARCH
45
40
35
30
25
20
15
10
5
0
10
20
30
40
Best Case
Average Case
Worst Case

Binary Search

Code :

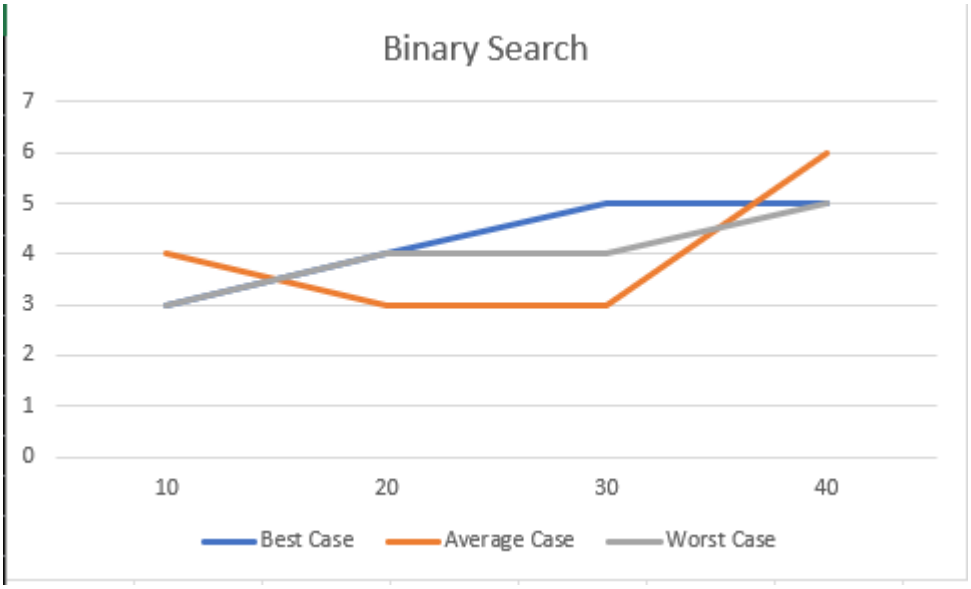
```
public class binnarysearch {

    public static long binarySearch(int[] arr, int key) {
        long comparisons = 0;
        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {
            comparisons++;
            int mid = (low + high) / 2;

            if (arr[mid] == key) {
                return comparisons;
            } else if (arr[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return comparisons;
    }

    public static int[] generateSortedArray(int n) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }
        return arr;
    }

    public static void main(String[] args) {

        int[] sizes = {10, 20, 30, 40};
```

```java
        for (int size : sizes) {
            int[] arr = generateSortedArray(size);

            System.out.println("Input Size: " + size);

            long best = binarySearch(arr, arr[size/2]);   // middle
element
            long worst = binarySearch(arr, -1);           // not
present
            long avg = binarySearch(arr, arr[size/3]);    // some
random position

            System.out.println("Best Case Comparisons: " + best);
            System.out.println("Average Case Comparisons: " + avg);
            System.out.println("Worst Case Comparisons: " + worst);
            System.out.println("----------------------------");
        }
    }
}
```

OUTPUT:

```
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>  & '
ssages' '-cp' 'C:\Users\yuvraj\AppData\Roaming\Code\User\work
b6392\bin' 'binnarysearch'
Input Size: 10
Best Case Comparisons: 3
Average Case Comparisons: 4
Worst Case Comparisons: 3
----------------------------
Input Size: 20
Best Case Comparisons: 4
Average Case Comparisons: 3
Worst Case Comparisons: 4
----------------------------
Input Size: 30
Best Case Comparisons: 5
Average Case Comparisons: 3
Worst Case Comparisons: 4
----------------------------
Input Size: 40
Best Case Comparisons: 5
Average Case Comparisons: 6
Worst Case Comparisons: 5
----------------------------
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>
```

GRAPH:


Binary Search
7
6
5
4
3
2
1
0
10
20
30
40
Best Case
Average Case
Worst Case

Factorial

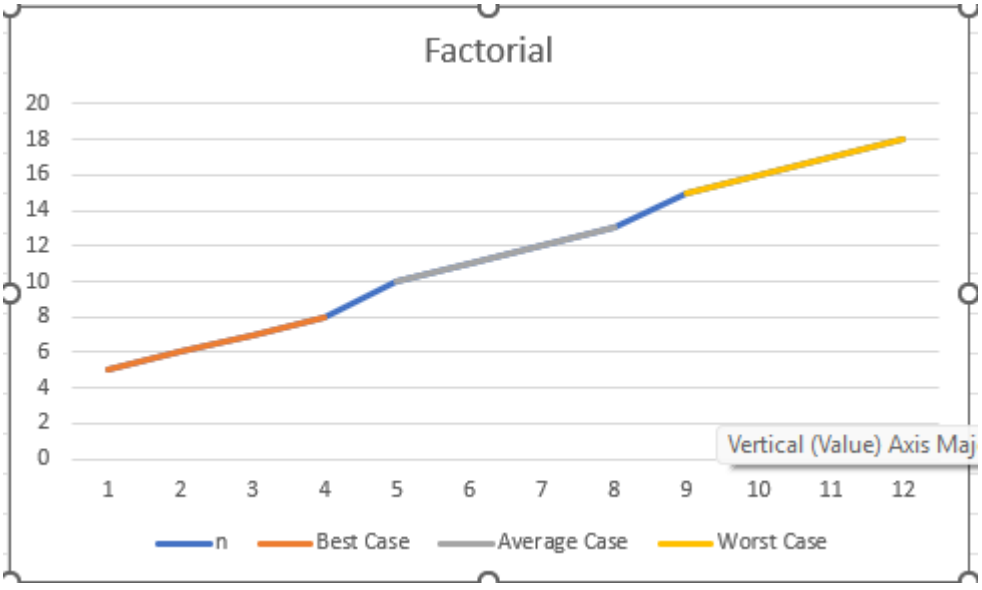
CODES:

```
public class FactorialCases {

    static int callCount = 0;

    public static long factorial(int n) {
        callCount++;
        if (n <= 1)
            return 1;
        return n * factorial(n - 1);
    }

    public static void runCase(int[] inputs, String caseName) {
        System.out.println(caseName);
        for (int n : inputs) {
            callCount = 0;
            factorial(n);
            System.out.println("n = " + n + " | Calls = " + callCount);
        }
        System.out.println("-----------------------");
    }

    public static void main(String[] args) {

        int[] bestCase = {5, 6, 7, 8};
        int[] avgCase = {10, 11, 12, 13};
        int[] worstCase = {15, 16, 17, 18};

        runCase(bestCase, "Best Case");
        runCase(avgCase, "Average Case");
        runCase(worstCase, "Worst Case");
    }
}
```

OUTPUT:

```
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>  & 'C:\Pr
ssages' '-cp' 'C:\Users\yuvraj\AppData\Roaming\Code\User\workspace
b6392\bin' 'FactorialCases'
Best Case
n = 5 | Calls = 5
n = 6 | Calls = 6
n = 7 | Calls = 7
n = 8 | Calls = 8
-----------------------
Average Case
n = 10 | Calls = 10
n = 11 | Calls = 11
n = 12 | Calls = 12
n = 13 | Calls = 13
-----------------------
Worst Case
n = 15 | Calls = 15
n = 16 | Calls = 16
n = 17 | Calls = 17
n = 18 | Calls = 18
-----------------------
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>
```

GRAPH:



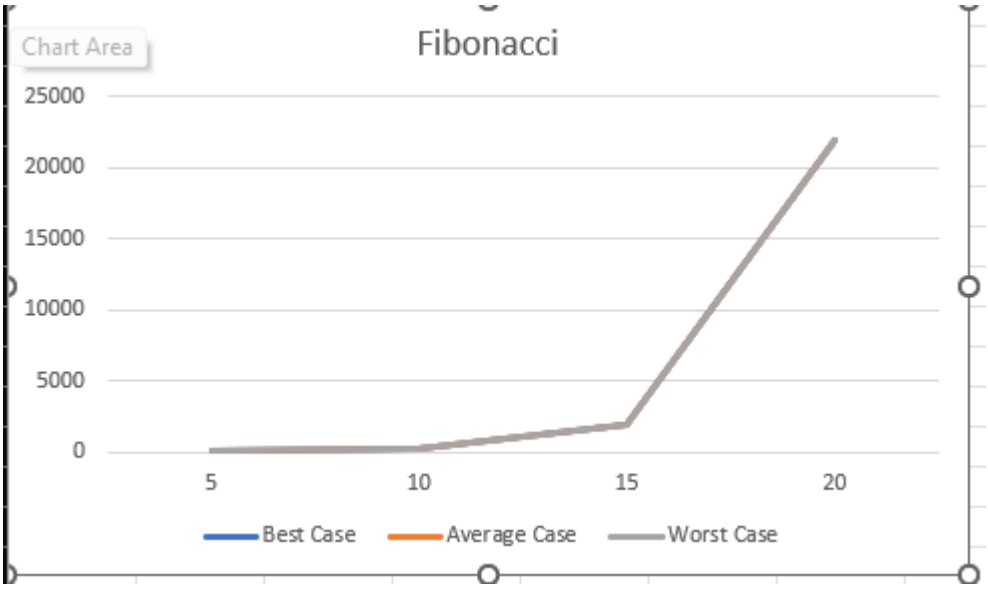
Fibonacci
CODE:

```java
public class fibonacci {

    static int callCount = 0;

    public static long fibonacci(int n) {
        callCount++;
        if (n <= 1)
            return n;
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    public static void runCase(int[] inputs, String caseName) {
        System.out.println(caseName);
        for (int n : inputs) {
            callCount = 0;
            fibonacci(n);
            System.out.println("n = " + n + " | Calls = " + callCount);
        }
        System.out.println("---------------------------");
    }

    public static void main(String[] args) {

        int[] bestCase = {5, 6, 7};
        int[] avgCase = {10, 11, 12};
        int[] worstCase = {20, 21, 22};

        runCase(bestCase, "Best Case");
        runCase(avgCase, "Average Case");
        runCase(worstCase, "Worst Case");
    }
}
```

OUTPUT:

```
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>  & 'c
ssages' '-cp' 'C:\Users\yuvraj\AppData\Roaming\Code\User\works
b6392\bin' 'FibonacciSimple'
Best Case (Small n)
n = 5 | Calls = 15
n = 10 | Calls = 177
n = 15 | Calls = 1973
n = 20 | Calls = 21891
--------------------------
Average Case (Medium n)
n = 5 | Calls = 15
n = 10 | Calls = 177
n = 15 | Calls = 1973
n = 20 | Calls = 21891
--------------------------
Worst Case (Large n)
n = 5 | Calls = 15
n = 10 | Calls = 177
n = 15 | Calls = 1973
n = 20 | Calls = 21891
PS C:\Users\yuvraj\Desktop\ADA_SEMESTER_4\LAB CODES ADA>
```

GRAPH:



 # same graph for all cases